

基于 LabVIEW 的语音识别与声卡音频信号 分析系统设计

作者姓名： 陈思羽

指导教师： 杨钢

单位名称： 信息科学与工程学院

专业名称： 测控技术与仪器

东 北 大 学

2026 年 5 月

课程设计任务书

课程设计题目：

基于 LabVIEW 的语音识别与声卡音频信号分析系统设计

设计的基本内容：

- (1) 利用电脑声卡采集外部语音或音频信号，实现音频信号实时采集。
- (2) 对采集到的音频信号进行 FFT 频谱分析，实时显示音频信号的时域波形与频域频谱。
- (3) 计算音频信号的幅值、主频、均方根值和信噪比等参数。
- (4) 设计音频滤波功能，实现低通、高通、带通等滤波方式，用于语音信号降噪。
- (5) 设计声卡输出测试音功能，生成特定频率和幅值的正弦波信号。
- (6) 调用 Windows/.NET 语音识别组件，实现简单语音命令控制。
- (7) 当语音输入“分析”时，自动调用音频分析子 VI。
- (8) 可扩展实现音调识别、音乐播放器或外部音乐软件调用等功能。
- (9) 设计用户登录功能，通过 Access 数据库保存用户名和密码，登录验证通过后方可进入主面板。

课程设计专题部分：

题目：基于 LabVIEW 的语音识别与声卡音频信号分析系统设计

设计或论文专题的基本内容：

本课题以 LabVIEW 为软件开发平台，利用计算机声卡、麦克风和扬声器作为音频输入输出设备，设计一个集语音命令识别、声卡音频采集、实时波形显示、频谱分析、参数计算、音频滤波和测试音输出于一体的虚拟仪器系统。

学生接受课程设计题目日期

第 9 周

指导教师签字:

2026 年 5 月 11 日

摘要

虚拟仪器技术将计算机软件与通用硬件设备结合,可以在较低成本下实现灵活的测量、分析和控制功能。声卡作为计算机常见的音频采集与输出设备,具有使用方便、成本低、实时性较好等特点,适合用于课程设计中的音频信号采集与处理实验。本文设计了一套基于 LabVIEW 的语音识别与声卡音频信号分析系统,系统以电脑麦克风作为输入设备,利用 LabVIEW 声音采集 VI 获取实时音频信号,并对采集数据进行时域显示、FFT 频谱分析、幅值计算、主频检测和信噪比估计。

系统同时调用 Windows 提供的.NET 语音识别组件`System.Speech.Recognition`,通过`SpeechRecognitionEngine`、`Choices`、`GrammarBuilder`和`Grammar`等类构建限定词语法,实现对“你好”“分析”“测试”“音乐”等简单语音命令的识别。当识别到“分析”命令时,主 VI 调用声卡音频信号分析仪子 VI,实现语音控制下的功能跳转。音频分析子 VI 内部采用循环采集与实时分析结构,每一帧采集数据均进行波形显示、频谱计算和参数更新。系统还设计了 Butterworth 音频滤波模块,可实现低通、高通和带通滤波,对语音信号进行降噪处理,并通过滤波前后波形和频谱对比展示滤波效果。此外,系统扩展了正弦波测试音输出、音调识别和音乐播放器控制等功能。

通过本次设计,完成了声卡音频采集、语音命令识别、频谱分析、音频滤波及测试音输出等模块的综合实现。系统前面板界面直观,功能模块划分清晰,能够较好地体现 LabVIEW 在信号采集、信号处理、人机交互和虚拟仪器系统集成方面的特点。

关键词: LabVIEW; 语音识别; 声卡采集; FFT; 音频滤波

目录

课程设计任务书	I
摘要	III
第一章 绪论	1
1.1 课题研究背景	1
1.2 课题研究意义	1
1.3 本文主要研究内容	2
第二章 系统总体方案设计	2
2.1 系统功能要求	2
2.2 总体设计思路	2
第三章 系统程序设计	4
3.1 语音识别程序设计	4
3.1.1 前面板	4
3.1.2 程序框图	5
3.1.3 .NET 管理	7
3.2 登录与权限控制程序设计	7
3.2.1 前面板	7
3.2.2 程序框图	8
3.3 音频采集程序设计	9
3.3.1 前面板	9
3.3.2 程序框图	10
3.3.3 信号处理	11
3.4 音频测试程序设计	12
3.4.1 前面板	12
3.4.2 程序框图	13
3.5 子 VI 设计	14
3.5.1 GoToVI 设计	14
3.5.2 参数配置模块子 VI 设计	14

3.5.3 数据库子 VI 设计	15
第四章 程序测试	17
4.1 登录功能测试	17
4.2 语音识别测试	17
4.3 声卡采集/播放测试	18
第五章 结束语	19
5.1 总结	19
5.2 心得体会	19
参考文献	21
附录	22
附录 A 主要 VI 列表	22
附录 B 主要参数设置	22
附录 C 核心公式	22

第一章 绪论

1.1 课题研究背景

随着计算机技术、数字信号处理技术和虚拟仪器技术的发展，传统仪器逐渐向软件化、模块化和智能化方向发展。虚拟仪器利用计算机强大的运算能力和灵活的软件界面，将数据采集、信号处理、显示分析和控制功能集成在统一平台中。LabVIEW 作为典型的图形化虚拟仪器开发平台，具有数据流编程直观、界面设计方便、硬件接口丰富等优点，在测控技术、自动化、信号处理和实验教学中得到了广泛应用。

语音信号是人机交互中最自然的输入方式之一。随着 Windows 系统和 .NET 平台对语音识别功能的支持，利用计算机自带麦克风实现简单语音命令识别已经具有较好的可行性。对于课程设计而言，通过 LabVIEW 调用 Windows 的语音识别组件，不仅可以实现简单命令控制，还能够加深对 .NET 调用、事件回调和状态控制等程序结构的理解。

另一方面，电脑声卡本质上是一种音频范围内的数据采集与输出设备。虽然声卡的采样范围和精度有限，但对于语音信号、正弦波测试信号和一般音频信号分析而言已经足够。利用声卡进行音频采集，并在 LabVIEW 中完成时域波形显示、FFT 频谱分析、幅值和频率测量，可以较好地体现虚拟仪器“用软件定义仪器功能”的思想。

因此，本课题将语音识别与声卡音频分析结合，设计一个能够通过语音命令调用分析功能的音频信号分析系统。该系统既包含语音控制的人机交互功能，又包含音频采集、滤波和频谱分析等信号处理内容，具有较好的综合性。

1.2 课题研究意义

本课题的意义主要体现在以下几个方面。

第一，系统利用普通计算机声卡作为采集设备，不需要额外购买专业数据采集卡，降低了实验成本，提高了系统可复现性。通过麦克风输入即可获得实际音频信号，适合课程设计和教学演示。

第二，系统将语音识别与音频信号分析结合，通过语音命令调用子 VI，实现了较为自然的人机交互方式。与传统按钮操作相比，语音命令控制更直观，也体现了智能化虚拟仪器的发展方向。

第三，系统涵盖了时域分析、频域分析、滤波处理、参数计算和声卡输出等多个模

块，有助于综合理解信号采集与数字信号处理流程。通过实时显示波形和频谱，可以直观观察音频信号的频率成分及滤波效果。

第四，系统采用模块化设计，将语音识别、音频分析、滤波、测试音输出等功能分离为不同模块或子 VI，便于调试、扩展和维护，也符合工程程序设计的基本思想。

1.3 本文主要研究内容

本文围绕基于 LabVIEW 的语音识别与声卡音频信号分析系统展开，主要研究内容包括：

1. 系统总体结构设计，包括主 VI、语音识别模块、音频分析子 VI 和扩展功能模块的划分。

2. Windows/.NET 语音识别组件在 LabVIEW 中的调用方法，包括关键词语法构造、回调事件注册和识别结果处理。

3. 基于声卡的音频信号采集方法，包括采样率、采样点数、设备 ID 等参数设置。

4. 音频信号时域波形显示与 FFT 频谱分析方法。

5. 幅值、RMS、主频和 SNR 等音频参数的计算方法。

6. Butterworth 滤波器在语音降噪中的应用。

7. 正弦波测试音生成和声卡输出方法。

8. 系统测试过程、常见错误分析和改进方向。



图 1.1 Main 程序总体前面板

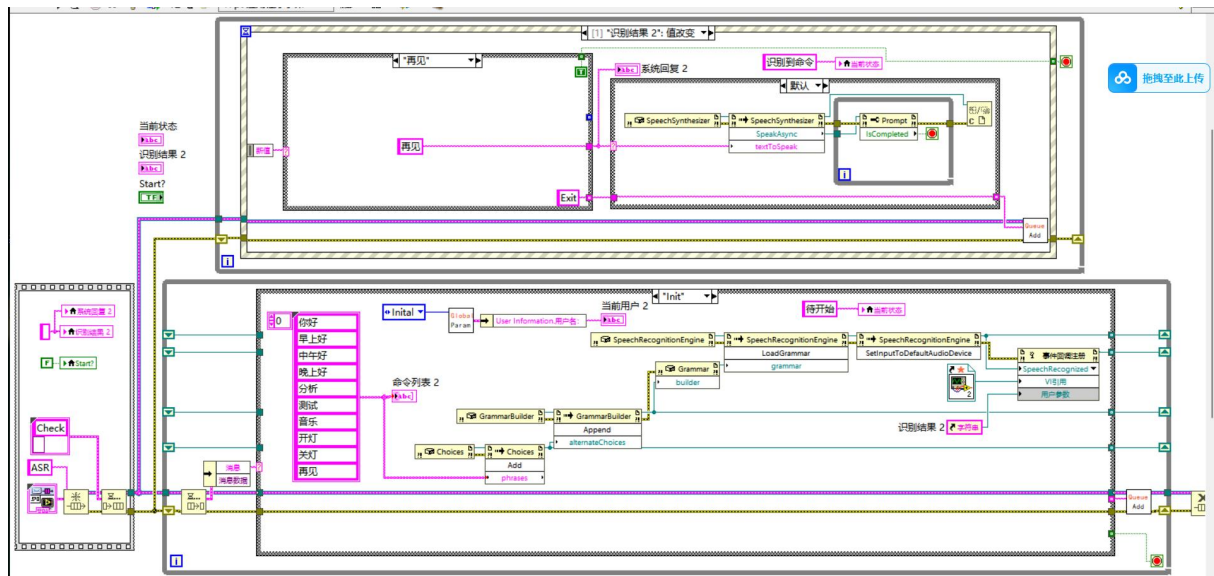


图 1.2 Main 程序框图(局部)

Callback	2026/5/12 14:40	文件夹	
Config	2026/5/13 12:39	文件夹	
Ctl	2026/5/12 13:59	文件夹	
Function	2026/5/13 0:27	文件夹	
Menu	2026/5/12 16:16	文件夹	
SubVI	2026/5/12 20:28	文件夹	
Tool	2026/5/12 11:28	文件夹	
Config.xml	2026/5/10 20:29	XML 源文件	1 KB
FinalAssignment.aliases	2026/5/13 14:00	ALIASES 文件	1 KB
FinalAssignment.lvpls	2026/5/13 14:00	LVLPS 文件	1 KB
FinalAssignment.lvproj	2026/5/13 14:00	LabVIEW Project	15 KB
Main.vi	2026/5/12 21:39	LabVIEW Instru...	35 KB

图 1.3 程序文件列表

第二章 系统总体方案设计

2.1 系统功能要求

根据课程设计要求 and 实际实现条件，本系统需要完成以下功能：

- (1) 语音命令识别功能。系统能够识别预设关键词，例如“你好”“早上好”“中午好”“下午好”“分析”“测试”“音乐”等，并根据识别结果执行对应操作。
- (2) 声卡音频采集功能。系统能够调用电脑麦克风，实时采集外部音频信号。
- (3) 时域波形显示功能。系统能够将采集到的音频数据以波形图形式实时显示。
- (4) 频谱分析功能。系统能够对采集音频进行 FFT 运算，显示频域频谱图。
- (5) 参数计算功能。系统能够计算当前音频帧的峰值幅值、RMS、主频和信噪比。
- (6) 音频滤波功能。系统能够实现低通、高通、带通滤波，并显示滤波后的波形和频谱。
- (7) 测试音输出功能。系统能够生成指定频率的正弦波，通过声卡输出用于测试。
- (8) 扩展功能。系统可根据语音命令打开音乐播放器或进行音调识别。

2.2 总体设计思路

系统采用“登录 VI + 主 VI + 功能子 VI”的结构。登录 VI 作为系统入口，负责用户身份验证和数据库查询；主 VI 负责语音识别初始化、语音命令接收以及对功能子 VI 的调用；音频信号分析仪子 VI 负责声卡采集、实时分析、滤波和测试音输出。

总体结构如下：

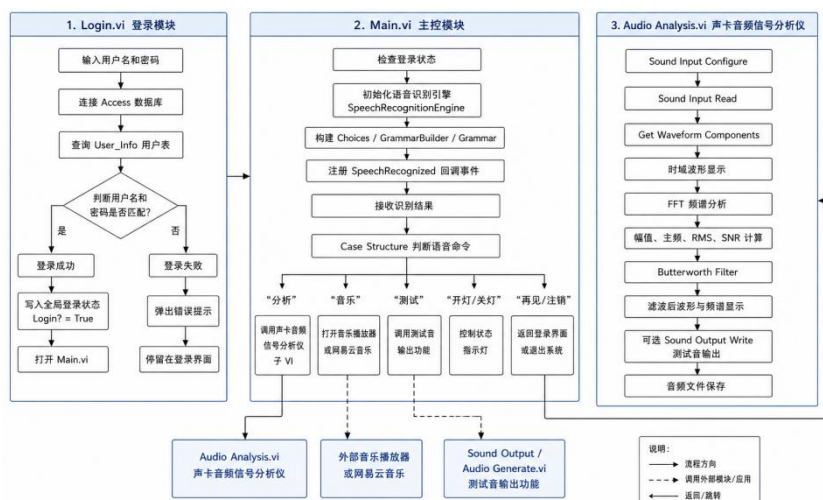


图 2.1 整体结构图

采用这种结构能够降低主 VI 的复杂度，使语音识别与信号分析之间保持清晰的功能边界。当后期需要修改音频分析算法或增加功能时，只需要修改分析子 VI，而不必大幅调整主 VI。

第三章 系统程序设计

3.1 语音识别程序设计

3.1.1 前面板

Main.vi 前面板设计为系统主控制界面，标题为“基于声卡的语音识别与音频分析系统”。



图 3.1 系统主控制界面

左侧为命令列表，列出“你好、早上好、中午好、下午好、分析、测试、音乐、开灯、关灯、再见”等可识别语音命令；中间区域显示识别结果和系统回复；下方显示当前用户并提供 Log Out 按钮；右侧使用圆形指示灯显示“灯”的开关状态。

为了防止用户直接打开 Main.vi 绕过登录，Main.vi 启动后首先读取 GlobalParameter 中的 Login? 状态和当前用户名。如果用户已经登录，则进入语音识别初始化流程；若未登录，则执行未登录分支，弹出“请先登录！”提示：



图 3.2 未登录弹窗

并通过 GoToVI 打开 Login.vi，同时关闭当前 Main.vi 前面板，从而防止用户直接打开 Main.vi 绕过登录。如图 3.1 所示，登录后，当前用户将显示当前登录用户。

3.1.2 程序框图

语音识别模块程序是通过 LabVIEW 的 .NET 节点调用 Windows 语音识别程序集。程序中主要使用 .NET Constructor Node、Invoke Node、Property Node 和 Register Event Callback 节点。

Main.vi 改进后采用生产者消费者结构实现。程序左侧首先创建消息队列，并向消息队列写入 Check 消息；上方为生产者循环，主要由事件结构组成，用于响应前面板按钮和识别结果值改变事件；下方为消费者循环，通过 Dequeue 取出消息并进入不同 Case 分支执行具体功能。生产者循环只负责“产生消息”，消费者循环负责“处理消息”，从而避免语音播报、打开子 VI 等耗时操作阻塞前面板事件响应。

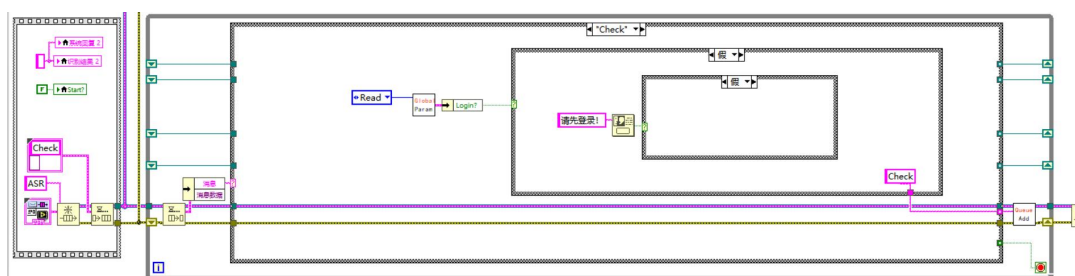


图 3.3 Check 分支

Main.vi 启动后首先执行 Check 消息分支，读取 GlobalParameter 中的 Login? 状态和当前用户名。如果用户已经登录，则向队列写入 Init 消息，进入语音识别初始化流程；若未登录，则执行未登录分支，弹出“请先登录！”提示，并通过 GoToVI 打开 Login.vi，同时关闭当前 Main.vi 前面板，从而防止用户直接打开 Main.vi 绕过登录。

在 Init 消息分支中，关键词字符串数组先接入 Choices.Add，再由 GrammarBuilder.Append 生成语法构造对象，随后创建 Grammar 并加载到 SpeechRecognitionEngine。识别引擎调用 SetInputToDefaultAudioDevice 使用默认麦克风输入。识别事件通过 Register Event Callback 连接到 Callback VI，用户参数传入“识别结果”字符串控件引用。

语音识别对象程序框图创建流程如下：

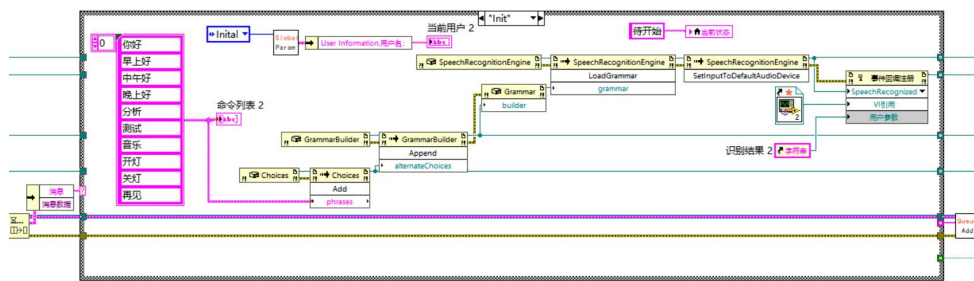


图 3.4 Init 消息分支

其中，Choices 用于保存可识别的关键词列表，GrammarBuilder 用于根据关键词构造语法规则，Grammar 是最终加载到识别引擎中的语法对象。由于本系统只需要识别有限的语音命令，因此采用限定词语法方式比开放式语音识别更稳定。

当识别引擎识别到语音命令后，会触发 `SpeechRecognized` 事件。LabVIEW 中通过 `Register Event Callback` 注册回调 VI，在回调 VI 中通过读取事件参数 `e.Result.Text` 得到识别文本，并将该识别文本字符串写入主界面的“识别结果”控件。

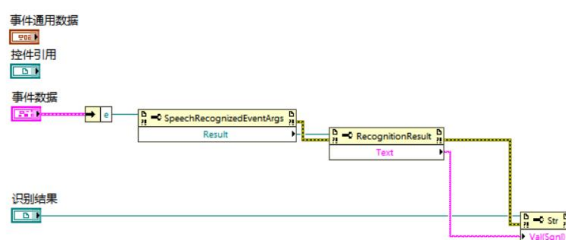
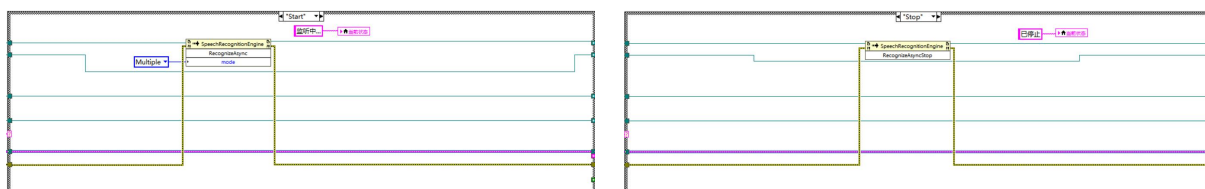


图 3.5 回调 VI 程序框图

为了使主 VI 中的“识别结果：值改变”事件能够被程序写值触发，Callback VI 使用的是字符串控件引用的 `Val(Sgnl)` 属性，而不是普通 `Value` 属性。这样当回调 VI 写入识别结果时，主 VI 的事件结构能够收到值改变事件，并继续执行回复、语音合成或功能跳转等逻辑。

语音识别的开始和停止分别由 `Start` 与 `Stop` 消息分支完成。`Start` 分支从移位寄存器中取出 `SpeechRecognitionEngine` 引用，调用 `RecognizeAsync(Multiple)` 启动异步连续识别，并将当前状态更新为“监听中”；`Stop` 分支调用 `RecognizeAsyncStop` 停止识别，并将当前状态更新为“已停止”。由于识别采用异步方式，`Start` 分支只需执行一次，识别引擎即可在后台持续监听，识别到语音后自动触发回调 VI。



图(a) Start 分支

图(b) Stop 分支

图 3.6 语音识别的开始与停止

`Main.vi` 的事件结构监听“识别结果”的值改变事件。识别结果变化后，程序进入对应字符串 `Case` 分支，根据命令生成系统回复并执行扩展操作。命令与系统回复对照表如表 2.1 所示：

表 2.1 命令与系统回复对照表

命令	系统
你好	你好啊
早上/中午/晚上好	早上/中午/晚上好呀
分析	已打开音频分析 VI
测试	已打开声卡输出测试音 VI
音乐	祝您听歌愉快！
开灯/关灯	已开灯/关灯
再见	再见！

例如，当识别结果为“分析”时，主 VI 使用 GoTo 子 VI 打开并运行声卡音频信号分析仪子 VI，同时将系统回复设置为“已打开音频分析 VI”，后调用 SpeechSynthesizer.SpeakAsync 播放系统回复，并通过 Prompt.IsCompleted 等待语音播报完成后关闭 .NET 引用。

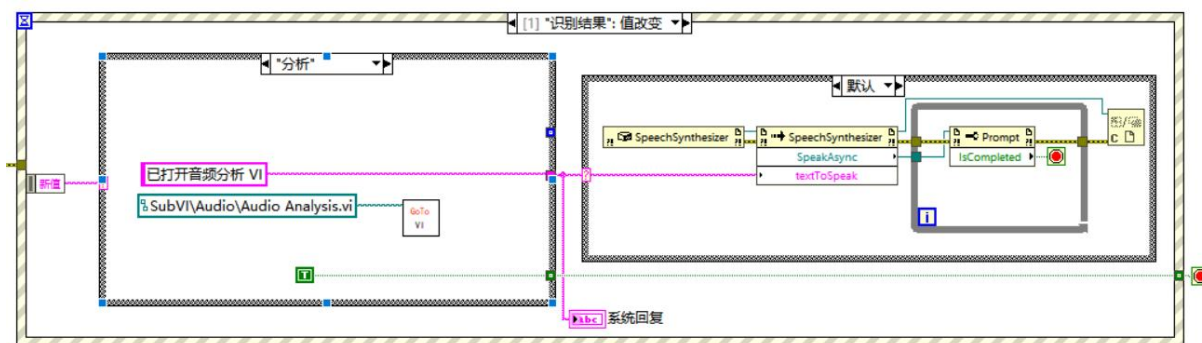


图 3.5 识别结果分支示意图

3.1.3 .NET 管理

LabVIEW 调用 .NET 对象时，会产生 .NET 引用。对于 SpeechRecognitionEngine、Grammar、GrammarBuilder 和 Choices 等由 Constructor Node 创建的对象，在程序结束或对象不再使用时，应使用 Close Reference 释放引用。

本 VI 中，SpeechRecognitionEngine 是最重要的对象，因为它占用麦克风输入和语音识别引擎资源。Choices 和 GrammarBuilder 虽然主要是内存对象，但在长期运行或多次初始化时也应释放，以避免引用累积。

3.2 登录与权限控制程序设计

3.2.1 前面板

本系统设计了独立的登录 VI 作为程序入口。登录界面包含用户名输入框、密码输入框、登录按钮和退出按钮。用户输入用户名和密码后，程序通过数据库连接字符串连接 Access 数据库，并在用户信息表中查询是否存在匹配记录。



图(a) 登陆前面板

图(b) 登陆失败

图 3.6 用户登录程序

Login.vi 前面板采用左侧登录区、右侧状态显示区的布局。左侧放置系统标题“登录界面”、用户名输入框、密码输入框、登录按钮和取消按钮；右侧放置数据库查询返回的 data 显示区和 Login 状态指示灯，用于调试登录结果。在 VI 运行时只显示左侧登录区，既能完成用户输入，又能直观调试登录验证是否成功。

3.2.2 程序框图

数据库连接字符串采用 OLE DB 方式，形式如下：

Provider=Microsoft.Jet.OLEDB.4.0;Data Source=data.mdb

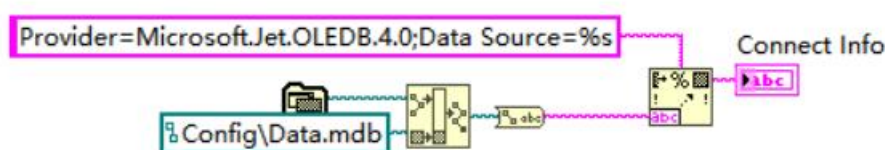


图 3.7 数据库连接字符串

其中，data.mdb 为 Access 数据库文件，用户信息保存在 User_Info 表中,如下表所示。

表 2 User_Info 表

User_Name	User_Password	User_Right
ccc	20051209	Administrator
Jack	12345678	User

登录验证 SQL 语句如下：

Select * from User_Info where User_Name = '%s' and User_Password = '%s'

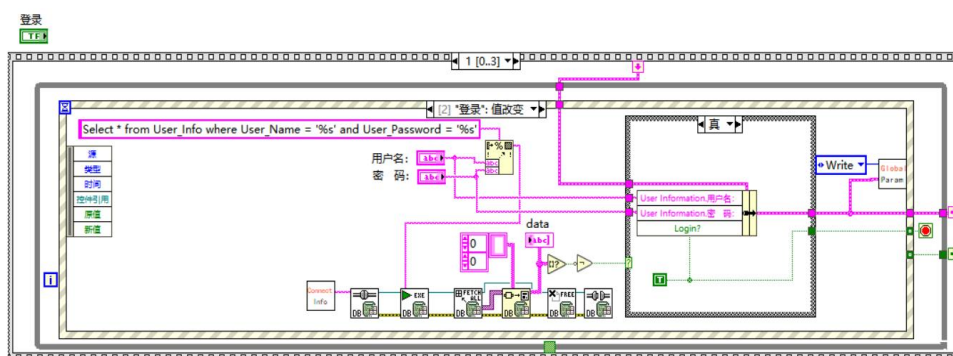


图 3.8 数据库查询账号密码

程序使用用户名和密码替换 SQL 语句中的占位符，然后通过 Database Connectivity 工具包执行查询。查询结果输出到 data 字符串数组中，程序通过判断返回数据是否为空来确定登录是否成功。若查询到匹配用户，则在 True 分支中将用户名、密码和 Login 状态写入 GlobalParameter 全局参数，登录成功。

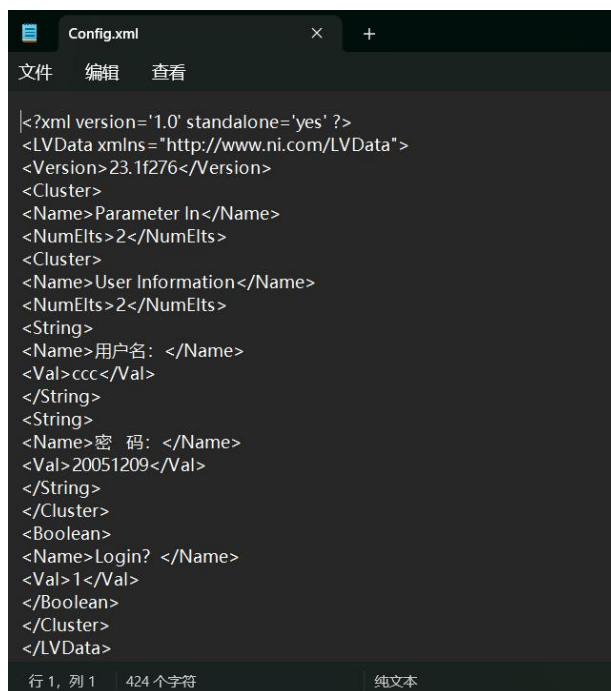


图 3.9 Config.xml 内容

如果查询结果为空，则说明用户名或密码错误，保持登录状态为 False，程序弹出提示信息，并停留在登录界面，如图 3.6 的图(b)所示。

登录成功后，GoTo 子 VI 通过 VI Server 打开并运行 Main.vi。

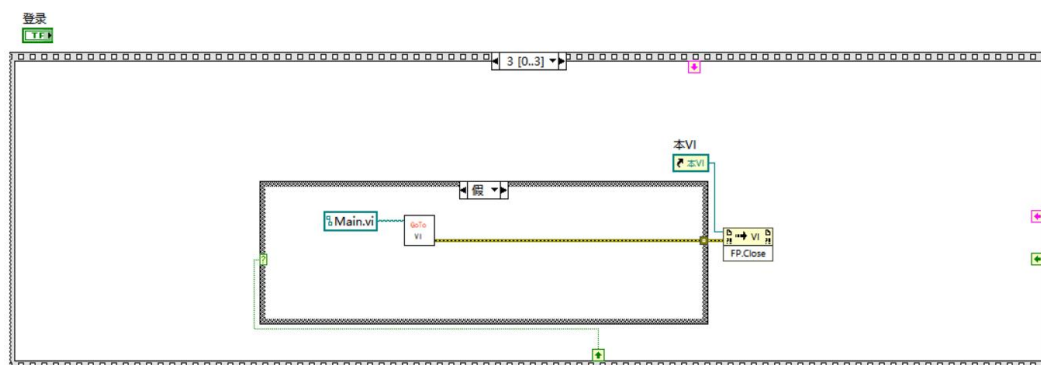


图 3.10 跳转帧

通过该设计，系统不仅能完成基本的用户身份验证，还能保证主功能界面必须在登录成功后才能使用。

3.3 音频采集程序设计

3.3.1 前面板

本 VI 以图表显示区和参数控制区分栏设计。左侧为主要显示区域，按照“滤波前”和“滤波后”分为上下两组，每组分别显示时域图和频谱图。右侧为控制与参数区域，包括设备设置、音频参数、滤波设置和音频文件保存地址。

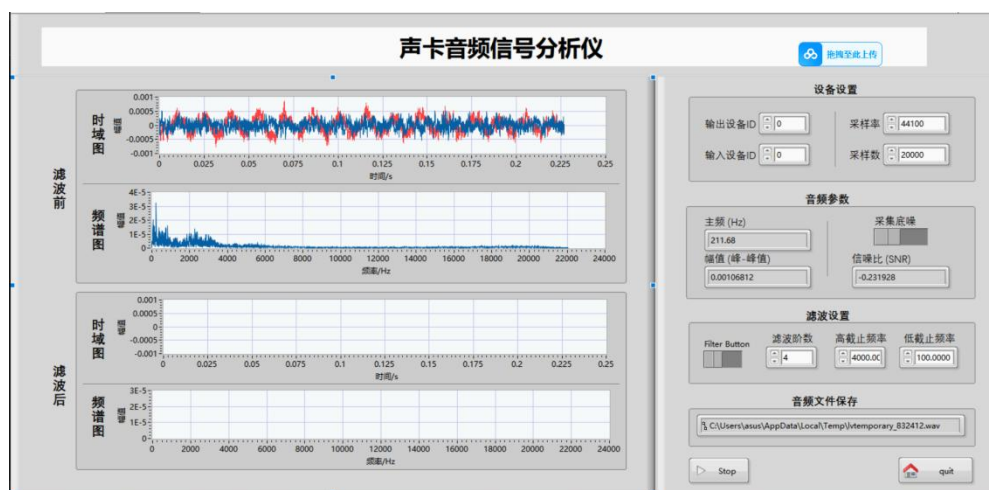


图 3.11 音频信号分析仪前面板

设备设置区包含输入设备 ID、输出设备 ID、采样率和采样数；音频参数区显示主频、峰峰值、采集底噪开关和信噪比；滤波设置区包含滤波开关、滤波阶数、高截止频率和低截止频率；底部设置 Stop 和 quit 按钮。该布局将实时显示和参数控制分开，便于观察信号变化和调节分析参数。

3.3.2 程序框图

音频采集模块使用 LabVIEW 声音输入函数实现。程序首先通过 Sound Input Configure 配置输入设备、采样率、采样点数和声道数。然后通过 Sound Input Read 启动采集。在 While Loop 中，程序周期性调用 Sound Input Read 读取一帧音频数据，并通过 Get Waveform Components 将 waveform 数据拆分为 Y 数组和 dt。

其中，Y 为音频幅值数组，dt 为相邻采样点之间的时间间隔，采样频率为：

$$f_s = \frac{1}{dt}$$

在实际设计中，采样率通常设置为 44100 Hz，单次采样点数设置为 4096。若采样点数过小，频率分辨率较低；若采样点数过大，实时刷新速度会变慢。因此本系统采用 4096 点作为较折中的设置。

实际 Audio Analysis.vi 采用消息队列结构实现。程序左侧首先创建消息队列，并向队列中写入 Init 消息。主循环由事件处理循环和消息处理循环组成：事件处理循环用于响应前面板按钮，消息处理循环根据队列中的消息执行不同状态。程序中主要消息包括 Init、Start、Acquire 和 Sound Out 等。

在 Start 分支中，程序根据前面板输入的采样率、采样数、输入设备 ID、声道数和位数等参数配置声卡输入，并准备音频文件保存路径。随后向队列写入 Acquire 消息，进入实时采集分析状态。该结构使采集、显示和退出控制更加清晰，也便于后续扩展更多功能状态。

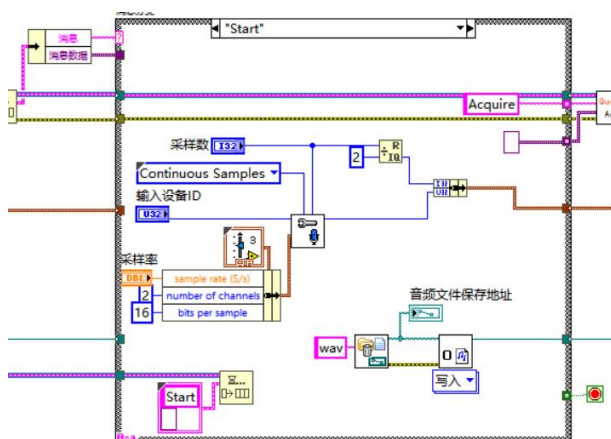


图 3.12 初始声音输入配置

在 **Acquire** 分支中，程序从声卡读取当前音频帧，并将采集到的声音数据显示到“原始信号”波形图；同时将采集数据写入 **wav** 文件，实现实时采集与文件保存同步进行。随后程序对同一帧音频数据进行幅值、频谱、主频、SNR 和滤波分析，完成音频信号分析仪的核心功能。

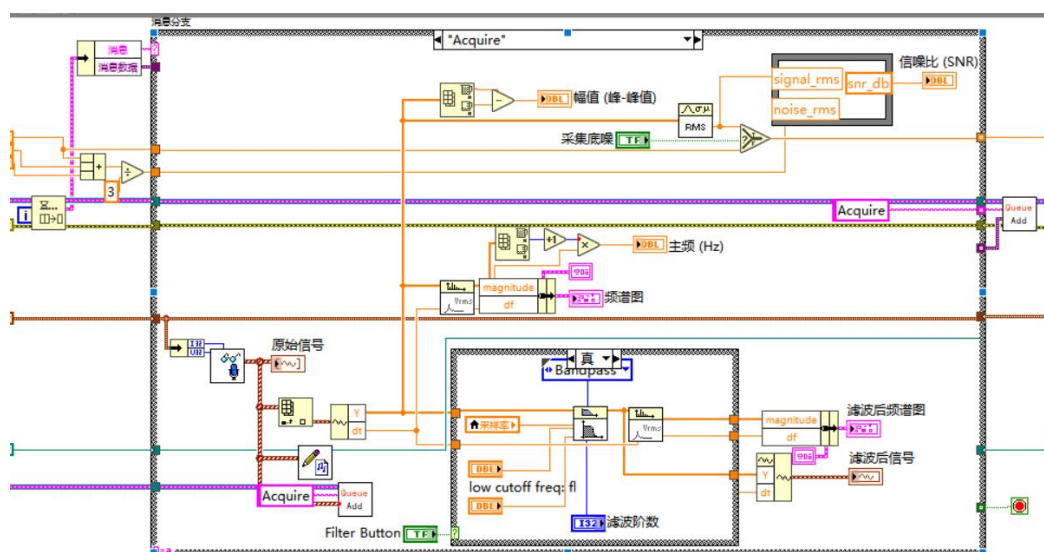


图 3.13 声音采集

3.3.3 信号处理

幅值计算、FFT 频谱分析以及音频滤波均可以使用 LABVIEW 自带的 VI 模块计算。我主要想在这里说明本 VI 的信噪比 SNR 计算。

信噪比用于表示信号强度与噪声强度的比值。由于实际语音输入中噪声难以完全分离，本系统采用噪声基准法进行估计。首先在安静环境下采集一段背景噪声，并计算其 RMS 值 $\text{Noise RMS} = \text{RMS}(\text{noise})$

实时采集过程中，对当前音频帧计算 RMS: $\text{Signal RMS} = \text{RMS}(\text{signal})$

信噪比计算公式为：

$$SNR(dB) = 20 \times \log_{10}\left(\frac{\text{Signal RMS}}{\text{Noise RMS}}\right)$$

为避免噪声 RMS 为 0 导致除法错误，可在分母中加入一个很小的常数：

$$SNR(dB) = 20 \times \log_{10}\left(\frac{\text{Signal RMS}}{\text{Noise RMS} + 10^{-6}}\right)$$

Formula Node 实际表达式：

$$SNR(dB) = 20 \times \log_{10}\left(\frac{\text{Signal RMS} + 10^{-6}}{\text{Noise RMS} + 10^{-6}}\right)$$

在实际程序实现中，系统在前面板设置“采集底噪”按钮。当该按钮为 True 时，程序将当前音频帧的 RMS 作为底噪采样值写入移位寄存器。为了避免单帧噪声波动造成 SNR 显示不稳定，系统使用三个移位寄存器保存最近三次底噪 RMS，并求平均值作为当前 Noise RMS：

$$\text{Noise RMS} = \frac{\text{Noise RMS1} + \text{Noise RMS2} + \text{Noise RMS3}}{3}$$

当“采集底噪”按钮为 False 时，三个移位寄存器不再写入新的信号 RMS，而是保持上一次采集到的底噪值。因此，系统在正常分析音频时仍使用之前保存的底噪平均值参与 SNR 计算。这样既能在需要时重新采集底噪，又能避免正常语音输入被误写入噪声基准。

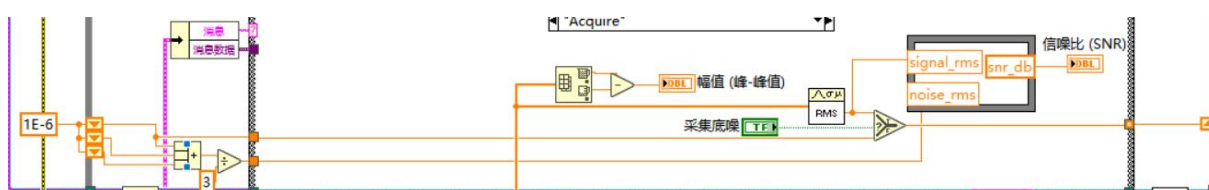


图 3.14 LABVIEW 程序实现

三个底噪移位寄存器的初值设置为 1E-6，用于避免程序刚启动时 Noise RMS 为 0。实际使用时，用户先保持环境安静，打开“采集底噪”开关，让程序采集几帧背景噪声；随后关闭该开关，系统便使用保存的底噪平均值进行实时 SNR 计算。

SNR 计算部分采用 Formula Node 实现。

该方法实现简单，适合课程设计实时显示。虽然它不能完全区分复杂环境下的语音和噪声，但可以较直观地反映当前输入音频相对于背景噪声的强弱变化。

3.4 音频测试程序设计

3.4.1 前面板

音频生成子 VI Audio Generate.vi 同样采用事件循环和消息处理循环结构。其前面板标题为“声卡输出测试音”，包含音量滑块、频率数值控件、设备 ID、波形显示图、Stop 和 Exit 按钮。

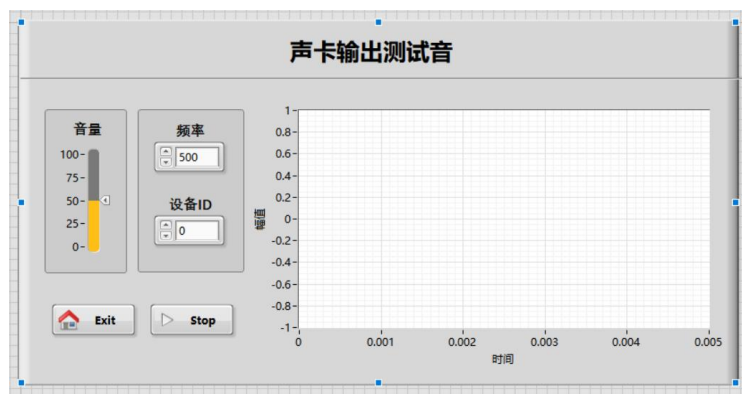


图 3.15 声卡输出测试前面板

3.4.2 程序框图

程序中事件循环将 Start、Flag、Quit 等操作写入消息队列；消息循环在 Start 分支配置声卡输出设备、采样率 44100 Hz、采样位数 16 bit 和连续采样方式；

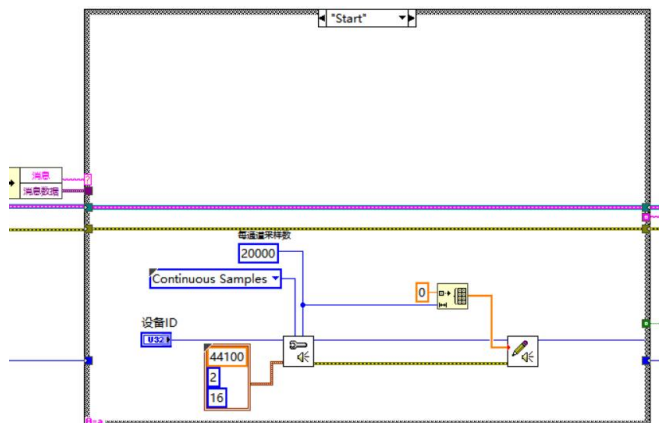


图 3.16 声音输出配置

在 Play 分支中，程序利用 Simulate Signal 生成 Sine 正弦波，并结合前面板频率和音量参数输出测试音。退出时通过 GoToVI 返回 Main.vi，并关闭当前前面板。

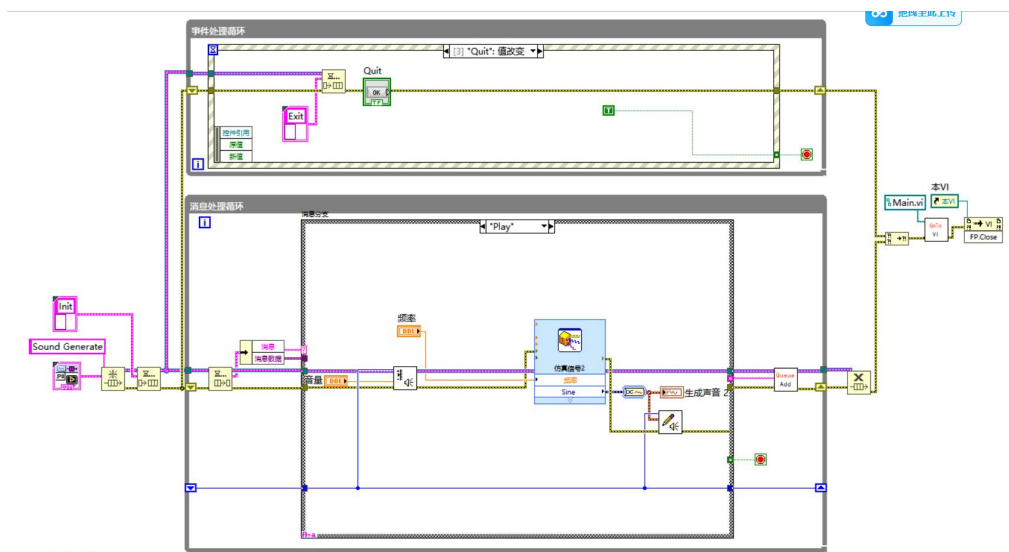


图 3.17 信号生成分支

3.5 子 VI 设计

3.5.1 GoToVI 设计

GoToVI 子 VI 用于统一处理界面跳转。该 VI 接收目标 VI 名称或相对路径，通过 Open VI Reference 打开目标 VI 前面板，调用 Run VI 运行目标界面，并将 Wait Until Done 设置为 False，使当前界面能够在启动目标界面后继续执行关闭流程。Login.vi、Main.vi、Audio Analysis.vi 和 Audio Generate.vi 的返回主界面或跳转功能均可复用该子 VI，减少重复框图。

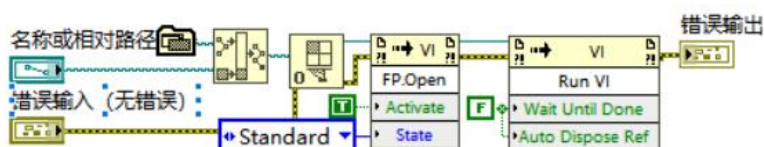


图 3.18 GoTo.vi 程序框图

3.5.2 参数配置模块子 VI 设计

配置参数模块由 ReadParameter.vi、SaveParameter.vi 和 GlobalParameter.vi 组成。

(1) ReadParameter.vi

ReadParameter.vi 从 Config.xml 中读取参数并转换为参数：

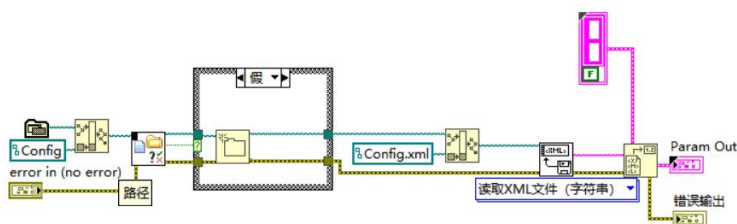


图 3.19 ReadParameter.vi 程序框图

(2) SaveParameter.vi

SaveParameter.vi 将当前参数簇重新写入 Config.xml：

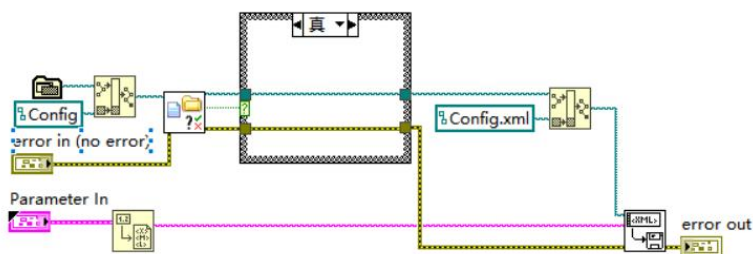


图 3.20 SaveParameter.vi 程序框图

(3) GlobalParameter.vi

GlobalParameter.vi 则通过读写分支保存登录用户信息、密码和登录状态。通过该设计，系统可以在不同 VI 之间共享登录信息，也便于保存和恢复系统配置。

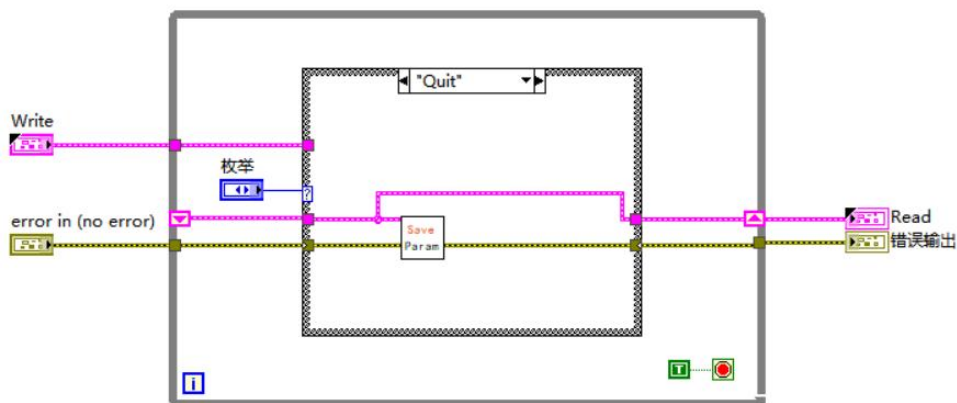


图 3.21 GlobalParameter.vi 程序框图

3.5.3 数据库子 VI 设计

数据库相关子 VI 由 Create Table.vi 和 Connect Information.vi 组成。

(1) Create Table.vi

Create Table.vi 用于在数据库里创建一张新表：

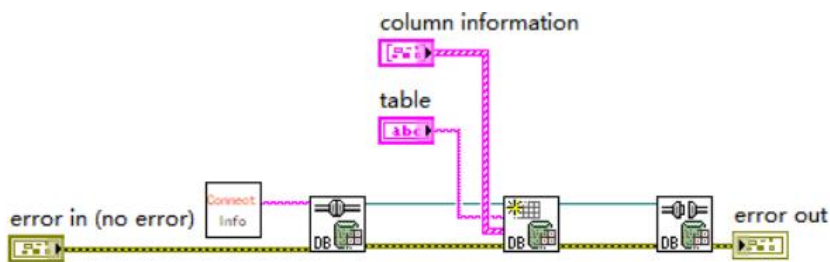


图 3.3.22 Create Table.vi 程序框图

Connect Information.vi 前面登陆界面已经介绍过了。

第四章 程序测试

4.1 登录功能测试

登录功能测试时，首先运行 Login.vi，输入数据库中已存在的用户名和密码，点击登录按钮。程序执行数据库查询后，若能正确跳转到 Main.vi，并在主面板显示当前用户信息，说明登录验证和界面跳转功能正常。

随后输入错误密码或不存在的用户名进行测试。此时数据库查询结果为空，程序应弹出“用户名或密码错误”等提示信息，并停留在登录界面，不允许进入主面板。该测试用于验证登录失败分支是否正确。



图(a) 登陆前面板

图(b) 登陆失败

图 4.1 用户登录测试

测试结果表明，登录模块能够根据数据库查询结果判断用户身份，并在登录成功后进入主面板，在登录失败或未登录状态下阻止主功能运行。

4.2 语音识别测试

语音识别模块测试时，首先运行主 VI，初始化语音识别引擎并加载关键词语法。随后对麦克风说出预设命令，如“你好”“下午好”“分析”“音乐”等。若系统正确识别，则识别结果显示区会更新对应文本，并触发相应 Case 分支。



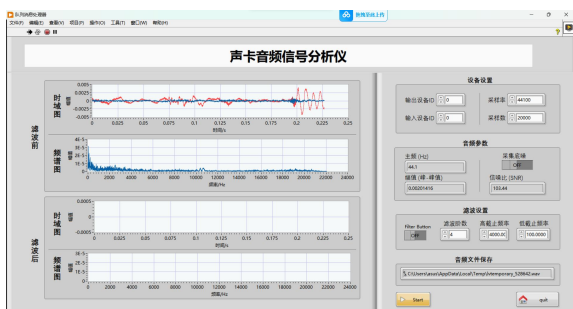
图 4.2 语音识别测试

测试结果表明，在安静环境下，限定词语法方式对简单中文短命令具有较好的识别效果。当关键词较短且发音清晰时，识别成功率较高。当环境噪声较大或麦克风距离较远时，可能出现识别延迟或识别错误。对此可通过减少关键词数量、提高麦克风输入质量和改善环境噪声来提高识别稳定性。

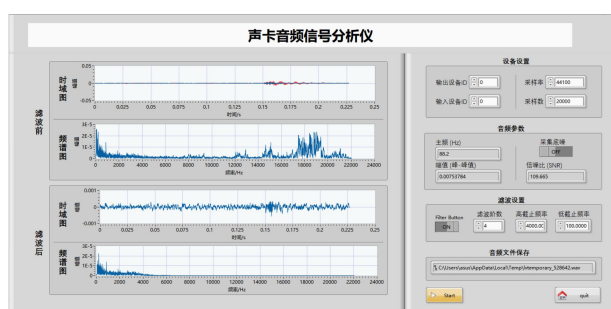
4.3 声卡采集/播放测试

运行声卡音频信号分析仪子 VI 后，设置输入设备 ID 为有效麦克风设备编号，点击 Start 开始采集。系统能够实时显示麦克风输入的时域波形。当环境安静时，波形幅值较小；当说话或播放声音时，波形幅值明显增大，说明声卡采集功能正常。

在测试中，采样率设置为 44100 Hz，采样点数设置为 4096 点。该设置下，系统刷新速度较快，波形显示较稳定，能够满足实时观察需求。



图(a) 滤波前



图(b) 滤波后

图 4.2 声卡采集测试

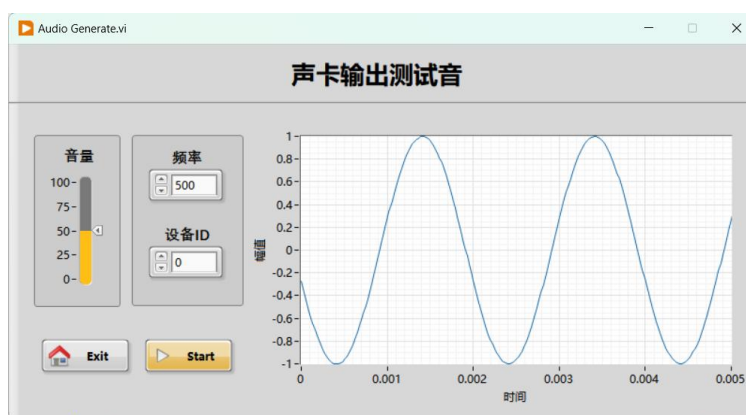


图 4.3 声卡输出测试

第五章 结束语

5.1 总结

本文设计并实现了一个基于 LabVIEW 的语音识别与声卡音频信号分析系统。系统首先通过 Login.vi 和 Access 数据库完成用户身份验证，登录成功后进入主面板；随后利用电脑麦克风采集音频信号，通过 LabVIEW 实现时域波形显示、FFT 频谱分析、幅值计算、主频检测和信噪比估计；同时利用 Butterworth 滤波器实现语音信号降噪，并通过正弦波测试音模块验证频谱分析功能。系统还调用 Windows/.NET 语音识别组件，实现了简单语音命令识别，并能够在识别到“分析”命令时调用音频分析子 VI。

从实现结果看，系统基本完成了课程设计要求。登录模块能够完成用户验证和主界面访问控制；前面板能够实时显示原始音频和滤波后音频的波形及频谱，参数计算结果能够随输入声音变化而更新，语音命令控制能够实现系统功能调用。该系统体现了 LabVIEW 在虚拟仪器设计中的优势，也展示了声卡作为简易数据采集设备在音频分析中的应用价值。

5.2 心得体会

通过本次课程设计，我对 LabVIEW 的图形化编程思想有了更深入的理解。与传统文本编程不同，LabVIEW 更强调数据流关系，程序执行顺序由数据依赖决定。因此，在设计程序时不仅要关注功能模块本身，还要注意数据线、错误线和引用线的连接顺序。尤其是在调用 .NET 对象和运行子 VI 时，引用管理和执行顺序非常重要。

在语音识别模块设计过程中，我学习了 LabVIEW 调用 .NET 程序集的方法，理解了 Constructor Node、Invoke Node、Property Node 和 Register Event Callback 的作用。语音识别不是简单调用一个函数，而是需要创建识别引擎、构造语法、加载语法、注册事件和异步启动识别。通过这一过程，我对事件驱动程序结构和回调函数有了更直观的认识。

在音频分析模块设计过程中，我进一步理解了声卡采集、采样率、采样点数、FFT、频率分辨率和滤波器参数之间的关系。实际调试中发现，很多理论公式在程序中都需要结合具体数据类型和显示方式进行处理。例如，频谱图不仅需要幅值数组，还需要正确的频率间隔；滤波器的截止频率必须与采样率匹配；输入设备 ID 和输出设备 ID 也不能混用。

本次设计也让我认识到模块化设计的重要性。将语音识别和音频分析拆分为不同子 VI 后，程序结构更清楚，调试更方便。后续如果继续完善系统，可以增加更稳定的

语音识别反馈、更完整的音乐播放器、更准确的基音检测算法，以及更友好的设备选择界面。

最后非常感谢杨老师能给我们这样一个动手实现的机会，以及老师辛勤的教学和指导。

参考文献

1. 陈锡辉, 张银鸿. LabVIEW 程序设计从入门到精通[M]. 北京: 清华大学出版社, 2007.
2. National Instruments. LabVIEW Sound Input and Sound Output VIs[EB/OL]. <https://www.ni.com/docs/>.
3. National Instruments. LabVIEW Signal Processing and FFT Analysis[EB/OL]. <https://www.ni.com/docs/>.
4. Microsoft. System.Speech.Recognition Namespace[EB/OL]. <https://learn.microsoft.com/dotnet/api/system.speech.recognition>.
5. Oppenheim A V, Schaffer R W. Discrete-Time Signal Processing[M]. 3rd ed. Upper Saddle River: Pearson, 2010.

附录

附录 A 主要 VI 列表

Main.vi: 系统主界面, 负责语音识别和功能调用。

Audio Analyzer.vi: 声卡音频信号分析仪子 VI。

Audio Generate.vi: 正弦波测试音生成子 VI。

Music Player.vi: 音乐播放或外部播放器调用子 VI。

Callback VI: 语音识别事件回调 VI。

附录 B 主要参数设置

采样率: 44100 Hz

采样点数: 20000

滤波类型: Bandpass

低截止频率: 100 Hz

高截止频率: 4000 Hz

滤波阶数: 4

测试音频率: 500 Hz

附录 C 核心公式

$$f_s = 1 / dt$$

$$df = f_s / N$$

$$\text{主频} = k * df$$

$$\text{峰值幅值} = \max(\text{abs}(x[n]))$$

$$\text{峰峰值} = \max(x[n]) - \min(x[n])$$

$$\text{RMS} = \sqrt{\text{mean}(x[n]^2)}$$

$$\text{SNR(dB)} = 20 * \log_{10}(\text{Signal RMS} / (\text{Noise RMS} + 1\text{E-}9))$$